

Node.js Events Module, Own HTTP Server & Auto Restart

FINAL Personal Revision Notes - Detailed Roman Urdu

Covered lesson: Events Module, EventEmitter, on(), emit(), data passing, event counter task, built-in HTTP module, createServer(), req/res, req.url, res.write(), res.end(), listen(), localhost/ports, Nodemon, package.json basics used in this practice, and node --watch.

1. Events Module ka basic concept

Node.js ka built-in **events** module event-driven programming ke liye hota hai. Simple socho: koi action hota hai, us action ka event naam hota hai, ek listener us event ka wait karta hai, aur jab event trigger hota hai to listener ka function run hota hai.

```
const EventEmitter = require("events");
const events = new EventEmitter();
```

new EventEmitter() array nahi banata. Ye EventEmitter class ka object/instance create karta hai jiske through hum events register aur trigger karte hain.

2. on() - event ko listen/register karna

```
events.on("login", (name) => {
  console.log(`Welcome Back ${name}`);
});
```

on() mein pehla argument event ka naam hota hai aur doosra listener/callback function. Is line se event trigger nahi hota; sirf listener ready hota hai.

3. emit() - event trigger karna

```
events.emit("login", "Muhammad Junaid Hassan");
```

emit() event ko fire karta hai. Event name ke baad jo values dete ho wo listener function ke parameters mein receive hoti hain.

Golden rule: on() = listen/register. emit() = trigger + data send.

4. Tumhara practical Events task

Tumne count object banaya jisme login, purchase, profile update aur logout ki counting rakhi. Phir har event ke listener mein related counter increment kiya aur end mein summary event se counts display kiye.

```
const count = {
  uselogin: 0,
  purchase: 0,
  update: 0,
  logout: 0
};
```

```
events.on("purchase", (name, item1, item2, count) => {
  console.log(`${name} you buy ${item1} and ${item2}`);
  count.purchase++;
});
```

```
events.emit("purchase", "Muhammad Junaid Hassan", "Computer", "Laptop", count);
```

Tumne login event ko loop mein do martaba emit bhi kiya. Is se ye clear hota hai ke same event multiple times trigger ho sakta hai aur counter har trigger par update ho sakta hai.

Earlier learning mistake

Pehle `on()` aur `emit()` ke roles reverse hue thay. Correct structure ye hai ke callback **`on()`** ke andar hota hai, jabke actual values/data **`emit()`** se bheje jate hain.

5. HTTP Module - apna server banana

Node.js ka built-in **http** module basic HTTP server create kar sakta hai. Is lesson mein Express ya kisi web framework ki zarurat nahi thi.

```
const http = require("http");
```

6. createServer()

```
const server = http.createServer((req, res) => {  
  // request receive hoti hai  
  // response bhejte hain  
});
```

`createServer()` server object create karta hai. Iske callback mein **req** aur **res** milte hain.

7. req vs res

req = browser/client se server ki taraf aane wali request. **res** = server se browser/client ko jane wala response.

```
res.write("WELCOME BACK JUNI");  
res.end();
```

Tumhari important correction ye thi ke response bhejte waqt **req.write()** nahi, **res.write()** use hona chahiye. Ab ye concept tumne clear kar liya hai.

8. req.url

```
if (req.url === "/") {  
  res.write("WELCOME BACK JUNI");  
  res.end();  
}
```

`req.url` requested path check karta hai. Is basic example mein `/` root/home path hai.

9. server.listen(), port aur localhost

```
const port = 4000;  
server.listen(port, () => {  
  console.log("Listening at 4000");  
});
```

`server.listen()` server ko given port par requests receive karne ke liye start karta hai. **localhost** apne computer ko refer karta hai. Example: `http://localhost:4000`.

10. Tumhara working OwnServer pattern

```
const http = require("http");  
  
const server = http.createServer((req, res) => {  
  if (req.url === "/") {  
    res.write("WELCOME BACK JUNI");  
    res.end();  
  }  
});  
  
const port = 4000;  
server.listen(port, () => {  
  console.log("Listening at 4000");  
});
```

Code cleanliness: Tumhari file mein variable name http bhi use hua tha. Agar consistently use karo to variable chal sakta hai, lekin professional readability ke liye **http** likhna better hai.

11. Normal server run

```
node OwnServer.js
```

Normal node command file run karta hai. Development mein changes ke baad automatic restart useful hota hai.

12. Nodemon - third-party development tool

Nodemon file changes watch karta hai aur save hone par Node process ko automatically restart karta hai.

```
npm install --save-dev nodemon  
npx nodemon OwnServer.js
```

Tumhare project ke package data mein Nodemon dependency add hui. Agar PowerShell direct **nodemon** command recognize na kare, project-local installation ke saath **npx nodemon OwnServer.js** straightforward method hai.

13. package.json ka is lesson mein role

package.json project ki basic information, scripts aur dependencies rakhta hai. Tumhare practice project mein CommonJS type, main file information, start script aur Nodemon dependency maujood thi.

```
"scripts": {  
  "start": "node server.js"  
},  
"dependencies": {  
  "nodemon": "^3.1.14"  
}
```

Filename note: server.js aur Server.js/OwnServer.js ko consistently name karna professional habit hai, specially case-sensitive systems ke liye.

14. Without third-party - node --watch

```
node --watch OwnServer.js
```

Ye Node ka built-in watch mode hai. Basic development workflow mein file change hone par process restart ho sakta hai aur Nodemon ki zarurat nahi hoti.

15. Final Quick Revision

Concept	Yaad rakhne wali line
EventEmitter	Events handle karne ka object/instance
on()	Listener register karta hai
emit()	Event trigger karta aur data bhej sakta hai
http.createServer()	Basic HTTP server create karta hai
req	Incoming request
res	Outgoing response
req.url	Requested path
res.write()	Response data bhejna
res.end()	Response finish karna

Concept	Yaad rakhne wali line
server.listen()	Port par server start/listen
Nodemon	Third-party auto restart
node --watch	Built-in auto restart

16. Self Test

- on() aur emit() ka difference apne words mein batao.
- emit() se 3 values listener ko kaise bhejoge?
- req aur res ka direction kya hai?
- Browser ko text bhejne ke liye req.write ya res.write?
- res.end() kyun use hota hai?
- server.listen(4000) ka kya matlab hai?
- Nodemon aur node --watch mein main difference kya hai?
- package.json mein scripts aur dependencies ka basic purpose kya hai?

Final lesson status

Events Module: DONE.

Basic HTTP Server: DONE at current learning stage.

Nodemon + built-in watch mode: DONE.

Advanced HTTP, Express, status codes, headers, streams aur advanced EventEmitter methods intentionally is PDF mein include nahi kiye gaye kyun ke ye abhi is lesson ka part nahi thay.